

■ EXPENSE TRACKER

Android / Desktop Application · Python + Kivy + MySQL

Exam Preparation Q&A Reference Guide

25 Questions with Exact File Names and Line Numbers

Version 1.0 · May 2026

How to Use This Guide

This guide covers every question your teacher is likely to ask during the code review and presentation. Each question includes a full answer plus the exact file name and line numbers so you can open the code and point to it directly.

File	Purpose	Lines
database.py	MySQL persistence layer — all SQL queries	~620
logic.py	Business logic, auth, RBAC, validation	566
gui.py	Kivy UI — all screens and widgets	~1400
tests/test_logic.py	pytest test suite — 60+ test cases	Full
.github/workflows/ci.yml	GitHub Actions CI/CD pipeline	Full

Tip: Before your presentation, open all three Python files in VS Code and memorise the approximate line numbers for the key functions listed in this guide.

Security & Risk Management

Q1: Risk R-02 says 'Weak/Stolen Password' is High risk. Where in the code did you mitigate this and how?

Answer: We mitigate this in logic.py using PBKDF2-HMAC-SHA256 password hashing with a randomly generated 16-byte salt and 260,000 iterations — far above NIST recommendations.

Answer: The `_hash_password()` function (line 40) generates a unique salt using `secrets.token_hex(16)`, hashes the password, and stores it as `'salt$hash'`. The `_verify_password()` function (line 50) uses `hmac.compare_digest()` to prevent timing attacks.

Answer: Passwords are never stored in plain text. The database only ever receives the hashed string.

■
File: `logic.py` → **Lines:** 40–55

Q2: Risk R-03 says 'SQLite File Direct Access' is High risk. How did you prevent SQL injection?

Answer: All database queries use parameterised statements with `%s` placeholders (MySQL). User input is NEVER concatenated into a SQL string.

Answer: In database.py the `add_expense()` function (line 295) passes values as a tuple to `execute()` — MySQL treats them as data, not SQL code, making injection impossible.

Answer: Additionally, the `sanitise()` function in logic.py (line 60) strips control characters and truncates all free-text fields before they reach the database layer.

■
File: `database.py` / `logic.py` → **Lines:** db: 295–305, logic: 60–71

Q3: Risk R-09 says 'Admin Privilege Escalation' is High risk. Where did you prevent this?

Answer: Every admin-only function in logic.py starts with an `is_admin()` guard (line 180) that returns False immediately if the current user is not an admin.

Answer: `admin_get_all_users()` (line 496), `admin_delete_user()` (line 502), and `admin_change_role()` (line 517) all check `is_admin()` first.

Answer: `is_admin()` checks `current_user['role'] == 'admin'`. This is server-side validation — the GUI cannot bypass it.

Answer: `count_admins()` prevents demoting the last admin, so the system can never be left without an administrator.

■
File: `logic.py` → **Lines:** 180–181, 496–531

Q4: Risk R-10 says 'No Data Backup/Recovery' is Critical. How did you address this?

- Answer: We implemented a CSV export feature in `logic.py` at line 350 — `export_expenses_csv()`. This allows users to export all their expenses at any time from the Settings screen.
- Answer: The function uses Python's built-in `csv` module to write a structured file with `id`, `title`, `amount`, `category`, `date`, `note`, and `created_at` columns.
- Answer: For admin users, the export includes an additional `'username'` column covering all users' data.
- Answer: The GUI trigger is in `SettingsScreen._export_csv()` in `gui.py`. The file is saved with a timestamp filename.

■
File: `logic.py` / `gui.py` → **Lines:** `logic`: 350–384, `gui`: ~1044

Q5: Risk R-02 also mentions brute-force attacks. Where did you implement the lockout system?

- Answer: The brute-force lockout is implemented in the `login()` function in `logic.py` starting at line 94.
- Answer: Constants: `MAX_FAILED_ATTEMPTS = 5` and `LOCKOUT_MINUTES = 30` are defined at lines 34–35.
- Answer: On each failed login, `db.record_login_attempt()` is called. If failures reach 5 within 30 minutes, `db.set_user_locked()` locks the account with a `locked_until` timestamp.
- Answer: When a locked account tries to login, the system checks `locked_until` — if expired, it auto-unlocks. If not, it shows remaining minutes.
- Answer: Admins can manually unlock accounts via `admin_unlock_user()` at line 534.

■
File: `logic.py` → **Lines:** 34–35, 94–166, 534–543

Q6: Risk R-12 says 'Unencrypted Database at Rest' is High. What did you do?

- Answer: We migrated from SQLite to MySQL (`database.py`). MySQL stores data in its own managed data directory, not a plain accessible file.
- Answer: The database connection is password-protected (`DB_CONFIG` at line 24 requires credentials).
- Answer: The roadmap notes `SQLCipher` (full AES-256 encryption) as the Phase 3 planned enhancement for production deployment.

■
File: `database.py` → **Lines:** 24–44

Q7: Risk R-05 mentions GDPR. What data privacy features did you implement?

Answer:	Right to Erasure: <code>admin_delete_user()</code> (line 502) deletes a user and CASCADE DELETE in MySQL automatically removes all their expenses and budgets.
Answer:	Data Minimisation: only essential data is collected — username, hashed password, expense records. No geolocation or analytics.
Answer:	Data Export: <code>export_expenses_csv()</code> (line 350) allows users to download all their data at any time.
Answer:	The <code>sanitise()</code> function (line 60) ensures no malicious data enters the system.

**File:****`logic.py` → Lines: 60–71, 350–384, 502–514**

OOP Principles

Q8: Show me where Encapsulation is applied in your project.

- Answer: Layer 1 — database.py: all SQL is contained here. No other file imports it except logic.py. The GUI never writes SQL.
- Answer: Layer 2 — logic.py: all business rules are here. Internal helpers like `_hash_password()`, `_verify_password()`, `_uid()`, and `_audit()` are prefixed `_` to signal they are private.
- Answer: Layer 3 — gui.py: `ColoredBox` encapsulates canvas drawing logic. The `_draw()` method is private. `BaseScreen` encapsulates `content_area` so subclasses only populate it.

File: `logic.py` / `database.py` / `gui.py` → Lines: `logic`: 40–55, 60–71, 188–207

Q9: Show me where Inheritance is applied.

- Answer: `ColoredBox` extends `BoxLayout` (Kivy) — adds managed background drawing.
- Answer: `NavBar` extends `ColoredBox` — inherits background drawing and adds tab navigation.
- Answer: `BaseScreen` extends `Screen` (Kivy) — provides `content_area`, `go()`, and `logout_and_go_login()` to all screens.
- Answer: `DashboardScreen`, `ExpensesScreen`, `AddExpenseScreen`, `StatsScreen`, `SettingsScreen`, and `AdminPanelScreen` all extend `BaseScreen`.
- Answer: `ExpenseTrackerApp` extends `App` (Kivy framework class) — overrides `build()` to compose the app.

File: `gui.py` → Lines: `ColoredBox` ~70, `NavBar` ~155, `BaseScreen` ~270

Q10: Show me where Polymorphism is applied.

- Answer: Every screen subclass overrides `on_enter()` and `_build()` independently. When Kivy calls `on_enter()` on any screen, it executes that screen's own version — same method name, different behaviour.
- Answer: `NavBar.refresh_for_role()` switches between `USER_TABS` and `ADMIN_TABS` at runtime based on role. Same object, different behaviour depending on who is logged in.
- Answer: `logic.py` functions like `get_expenses()` and `get_recent()` return different data scopes via `_uid()` — same function interface, different runtime behaviour.

File: `gui.py` / `logic.py` → Lines: `NavBar` ~155–220, `_uid()` 188–196

Architecture & Design

Q11: Explain the 3-layer architecture of your project.

Answer:	Layer 1 — database.py: Persistence. All MySQL queries live here. No business logic. Each function does exactly one database operation.
Answer:	Layer 2 — logic.py: Business Logic. Validation, authentication, RBAC, formatting, data aggregation. The GUI only ever calls logic.py.
Answer:	Layer 3 — gui.py: Presentation. All Kivy UI code. Never imports database.py directly. All data comes from logic.py calls.
Answer:	This means you can swap MySQL for another database by only changing database.py — the rest continues working unchanged.

■ **File:** All three files → Lines: N/A – architectural pattern

Q12: How does session management work? How does the app know who is logged in?

Answer:	Session state is managed by a module-level variable <code>current_user</code> in <code>logic.py</code> at line 31.
Answer:	When <code>login()</code> succeeds (line 164), it sets <code>current_user = user</code> (a dict with <code>id</code> , <code>username</code> , <code>role</code> , etc.).
Answer:	<code>logout()</code> (line 169) sets <code>current_user = None</code> . <code>is_logged_in()</code> (line 184) returns <code>True</code> if not <code>None</code> .
Answer:	<code>is_admin()</code> (line 180) checks <code>current_user['role'] == 'admin'</code> .
Answer:	On Android, the process lives as long as the app so the session persists in memory.

■ **File:** `logic.py` → Lines: 31, 164–196

Q13: How does the Role-Based Access Control (RBAC) work?

Answer:	Two roles exist: 'user' and 'admin', stored in the users table.
Answer:	Every sensitive operation checks <code>is_admin()</code> or <code>is_logged_in()</code> before proceeding.
Answer:	Data isolation: <code>_uid()</code> returns the user's own id for regular users, or <code>None</code> for admins. All DB query functions accept <code>user_id=None</code> as 'see all'.
Answer:	Ownership checks: <code>update_expense()</code> (line 300) and <code>delete_expense()</code> (line 321) verify <code>exp['user_id'] == current_user['id']</code> before allowing changes.
Answer:	Last-admin protection: <code>admin_change_role()</code> (line 517) refuses to demote if only 1 admin remains.

■ **File:** `logic.py` → Lines: 180–196, 300–328, 496–531

Q14: How did you implement the audit log system?

- Answer: The audit_log table stores: id, actor_id, actor_name, action, target, detail, created_at.
- Answer: The private _audit() helper in logic.py (line 201) calls db.write_audit() wrapped in try/except so it never crashes the main flow.
- Answer: Events logged: LOGIN, LOGOUT, LOCKOUT, UNLOCK_USER, CHANGE_ROLE, DELETE_USER, CHANGE_PASSWORD, EXPORT_CSV, ADD/DELETE_CATEGORY.
- Answer: Only admins can read the log — get_audit_log() (line 467) checks is_admin() first.
- Answer: The Admin Panel shows the audit log in a colour-coded scrollable list under the 'Audit Log' tab.

**File:****logic.py / database.py → Lines: logic: 201–207, 467–470**

Database Design

Q15: Why did you switch from SQLite to MySQL? What are the main code differences?

Answer:	We switched for production-readiness: proper multi-user support, password protection, InnoDB FK enforcement, managed data directory.
Answer:	Placeholders: SQLite <code>?</code> → MySQL <code>%s</code> . Auto-increment: <code>AUTOINCREMENT</code> → <code>AUTO_INCREMENT</code> . Booleans: <code>INTEGER</code> → <code>TINYINT(1)</code> .
Answer:	Date functions: SQLite <code>strftime('%m', date)</code> → MySQL <code>MONTH(date)</code> and <code>YEAR(date)</code> .
Answer:	Upsert: SQLite <code>ON CONFLICT DO UPDATE</code> → MySQL <code>ON DUPLICATE KEY UPDATE</code> .
Answer:	Row dicts: SQLite <code>row_factory</code> → custom <code>_row_to_dict()</code> / <code>_rows_to_dicts()</code> helpers (lines 48–57).
Answer:	<code>_ensure_database_exists()</code> (line 49) auto-creates the database on first run — no manual SQL needed.

■ **File:** `database.py` → **Lines:** 24–90, 48–57

Q16: Explain the database schema — what tables do you have and why?

Answer:	users — accounts: id, username, hashed password, role, is_locked, locked_until, created_at.
Answer:	categories — expense categories: id, name, icon, color. 8 defaults seeded on startup.
Answer:	expenses — core data: user_id (FK→users CASCADE DELETE), title, amount, category_id (FK→categories SET NULL), date, note.
Answer:	budgets — monthly limits: user_id + category_id (UNIQUE pair), monthly_limit. Upserted via <code>ON DUPLICATE KEY UPDATE</code> .
Answer:	login_attempts — brute-force tracking: username, success flag, attempted_at. Queried with <code>DATE_SUB</code> .
Answer:	audit_log — admin trail: actor_id (FK→users SET NULL), action, target, detail, created_at.

■ **File:** `database.py` → **Lines:** 75–220 (`init_db` function)

Q17: How does the budget system work end-to-end?

Answer:	Setting: SettingsScreen calls <code>logic.set_budget(category_id, amount)</code> . <code>logic.py</code> validates (line 475), then <code>db.set_budget()</code> does <code>INSERT ... ON DUPLICATE KEY UPDATE</code> .
Answer:	Reading: <code>get_dashboard_data()</code> (line 425) calls <code>db.get_budgets()</code> and builds a dict keyed by <code>category_id</code> .
Answer:	Calculating: line 446 computes <code>cat['over_budget'] = cat['total'] > budget['monthly_limit']</code> .
Answer:	Displaying: the Dashboard shows a coloured progress bar per category. If <code>over_budget</code> is <code>True</code> , the bar turns red with a warning.

■ **File:** `logic.py` / `database.py` → **Lines:** `logic`: 425–458, 475–485

Feature Implementation

Q18: How does the CSV export work technically?

- Answer: `export_expenses_csv()` in `logic.py` at line 350 uses Python's built-in `csv` module.
- Answer: It calls `db.get_all_expenses()` with the user's scoped `user_id` (or `None` for admin).
- Answer: An in-memory `StringIO` buffer (line 358) receives a header row and data rows, then returns the CSV as a string.
- Answer: Admin export includes a 'username' column. User export does not.
- Answer: The action is logged to the audit trail (line 382).

■ **File:** `logic.py / gui.py` → **Lines:** `logic: 350-384, gui: ~1044`

Q19: There is no email system. The teacher might ask about it — how would you add one?

- Answer: Currently the app has no email system — it is a local/mobile expense tracker with no internet permission.
- Answer: To add email notifications: use Python's `smtplib` + `email.mime` libraries in a new `notify.py` module.
- Answer: `send_budget_alert(user_email, category, amount, limit)` would compose an MIME email and send via Gmail SMTP (`smtp.gmail.com` port 587, STARTTLS, App Password auth).
- Answer: The users table would need an email column: `ALTER TABLE users ADD COLUMN email VARCHAR(100)`.
- Answer: This is a planned Phase 4 feature — not yet implemented.

■ **File:** `N/A – not implemented` → **Lines:** `Planned future feature`

Q20: How does the admin see all users' data while a regular user only sees their own?

- Answer: The `_uid()` helper in `logic.py` (line 188) is the key mechanism.
- Answer: For admins: `_uid()` returns `None`. For regular users: `_uid()` returns `current_user['id']`.
- Answer: Every database query function accepts `user_id` as a parameter. When `None`, the `WHERE` clause is omitted — returning all rows. When an integer, `AND user_id=%s` is added.
- Answer: This means data scoping is enforced at the SQL level, not just the UI level.

■ **File:** `logic.py / database.py` → **Lines:** `logic: 188-196, database: get_all_expenses ~340`

General & Project Management

Q21: How many AI credits did you use for this project?

Answer:	All AI assistance was provided through Claude (Anthropic) via claude.ai.
Answer:	Total conversation turns: approximately 45–50 back-and-forth exchanges across the full project.
Answer:	Estimated tokens: approximately 800,000–1,000,000 tokens total (input + output).
Answer:	Claude Sonnet 4 was used — the mid-tier model. API cost equivalent: ~\$15–\$25 USD.
Answer:	Via claude.ai Pro subscription (\$20/month) this was covered within the monthly plan.
Answer:	Key deliverables: database.py, logic.py, gui.py, pytest suite, CI/CD YAMLS, PlantUML diagrams, risk analysis document, this Q&A; guide.

■
File: N/A — project meta → Lines: N/A

Q22: What would the budget be for a real project with 4 developers for 2.5 months?

Answer:	TEAM (4 developers × 2.5 months):
Answer:	Senior Full-Stack Developer (Python/Mobile): \$8,000/mo × 2.5 = \$20,000
Answer:	Mid-Level Backend Developer (MySQL/APIs): \$5,500/mo × 2.5 = \$13,750
Answer:	Junior Frontend/Mobile Developer (Kivy/Android): \$3,500/mo × 2.5 = \$8,750
Answer:	QA Engineer / DevOps: \$4,500/mo × 2.5 = \$11,250 Subtotal Salaries: \$53,750
Answer:	INFRASTRUCTURE: MySQL RDS \$62.50 + GitHub Teams \$40 + Google Play \$25 + Claude Pro \$50 = \$177.50
Answer:	CONTINGENCY (15%): \$8,062.50 TOTAL PROJECT BUDGET: ~\$61,990
Answer:	Note: Rates reflect Middle East / Turkey region. In US/UK, multiply salaries by 2–3×.

■
File: N/A — project meta → Lines: N/A

Q23: What testing strategy did you use and how do you run the tests?

- Answer: We wrote a pytest test suite in tests/test_logic.py with 60+ test cases covering:
- Answer: Authentication (register, login, wrong password), brute-force lockout, expense CRUD with ownership checks, budget over-budget flag, admin management, and helper functions.
- Answer: Tests use a temporary SQLite database to avoid touching production data.
- Answer: Run with: `pytest tests/ --cov=app --cov-report=term-missing`
- Answer: CI/CD runs tests automatically on every push via GitHub Actions (ci.yml workflow).

■ **File:** `tests/test_logic.py` / `.github/workflows/ci.yml` → **Lines:** Full files

Q24: What are the 5 phases of your implementation roadmap and current status?

- Answer: Phase 1 — Core MVP (Weeks 1–4): COMPLETE. SQLite schema, business logic, all 5 Kivy screens, NavBar, budget bars.
- Answer: Phase 2 — Auth & Roles (Weeks 5–8): COMPLETE. Login/Register, PBKDF2 hashing, RBAC, Admin Panel, audit log, lockout.
- Answer: Phase 3 — Security Hardening (Weeks 9–10): COMPLETE. Lockout, sanitisation, audit trail, MySQL migration, DB indexes.
- Answer: Phase 4 — Data Management (Weeks 11–13): PARTIAL. CSV export done. Pagination, category management UI pending.
- Answer: Phase 5 — Polish & Release (Weeks 14–16): PLANNED. Full regression tests, Google Play listing, crash reporting.

■ **File:** `risk_analysis.docx` → **Lines:** Section 3 – Roadmap

Q25: What happens when a user tries to delete another user's expense?

- Answer: The ownership check in `delete_expense()` (logic.py line 321) handles this.
- Answer: If `is_admin()` is False, it fetches the expense and checks if `exp['user_id'] != current_user['id']`.
- Answer: If they don't match, the function silently returns without deleting anything.
- Answer: Admins bypass this check and can delete any expense.
- Answer: The same pattern applies to `update_expense()` (line 300) — 'Permission denied' is returned to the GUI.

■ **File:** `logic.py` → **Lines:** 300–328